

ARBITER — Mid-Semester Progress Report

Adaptive Multi-Agent System for Iterative Task Generation, Validation and Refinement Gautam Lasgotra (23BCS032) · Chirag Attri (23BCS024) · Aniket Kundal (23BCS015) Guide: Dr. Sonika Gupta, SoCSE · SMVDU Katra · **3 October 2026** Repository: `github.com/igautamlasgotra/arbiter` · Live demo: `arbiter-gules-two.vercel.app`

1. Status summary

The core system is **built, deployed and measured**. An unseen task typed at runtime is answered by a real multi-agent loop: tests are designed, a solution is generated, the solution is executed against those tests, failures are fed back, and the solution is revised until it passes or the budget stops it. The loop, the validators, the budget governor, all five baseline conditions and the live web interface are complete and covered by **42 automated tests that run offline**. The demo is hosted publicly so it can be opened from any machine.

The first real measured runs are in Section 3. They are from a five-task smoke set, not the final benchmark, and Section 3.2 states plainly what they can and cannot support.

Area	Status
Data contracts, run state, budget governor	✔ Complete
LLM provider abstraction, response cache, multi-key router	✔ Complete
Sandboxed code execution	✔ Complete
Validators — executable tests, answer matching	✔ Complete
Agent role library — generator, repair, critic	✔ Complete
Test-designer agent (enables unseen runtime tasks)	✔ Complete
Refinement loop with bounded termination	✔ Complete
Baselines A, B, C	✔ Complete
Baseline A+ (budget-matched single agent)	✔ Complete
Adaptive planner (condition D)	✔ Implemented, not yet evaluated
Live web interface with execution trace	✔ Complete
Replay mode (demo safety net)	✔ Complete
Metrics module	✔ Complete
Resumable experiment runner	✔ Complete
Live API integration (Gemini), measured token accounting	✔ Complete
Public deployment (Vercel), token-gated live runs	✔ Complete
Benchmark datasets — code / SQL / math	🕒 Smoke set only; full sets in Phase 2
Experimental results	🕒 Smoke set measured; frozen benchmark in Phase 3

2. Verified end-to-end behaviour

The following is a **real, unedited** event stream from the system, produced by running an unseen task through the HTTP interface. It is the exact behaviour described in the project review: one agent produces work, another validates it, feedback returns, the work is revised.

```
event: design_tests    role=test_designer    entry=solve, 2 tests
event: plan            roles=["generator"]
event: generate         role=generator        iteration 1
event: validate         role=python_exec      score=0.50  passed=false
event: decide          refine
event: generate         role=generator        iteration 2
event: validate         role=python_exec      score=1.00  passed=true
event: decide          accept
event: stop            accepted
```

The first attempt was wrong, the executed tests caught it (1 of 2 tests passing → score 0.50), the failure was fed back, and the second attempt passed. Total: 3 LLM calls, 2 iterations. Two details worth noting to the panel:

- **Validation is not an opinion.** `python_exec` runs the generated program in a sandboxed subprocess and reports which named tests failed. There is no LLM in that path. This matters because Huang et al. (ICLR 2024) showed refinement without external feedback does not help.
- **The score is graded, not binary.** 0.50 means one of two tests passed. Without partial credit the system could not tell "getting closer" from "stuck", and the no-improvement stop condition would be meaningless.

3. First measured results

All five conditions were run against the five-task smoke set with the real Gemini API. **25 task-runs, every number below computed from the logged traces** (`traces/smoke.jsonl` , committed to the repository).

Condition	n	Pass rate	Mean tokens	Mean LLM calls
A — single agent	5	100%	240	1.0
A+ — budget-matched single agent	5	100%	431	1.6
B — generator ↔ validator loop	5	100%	240	1.0

Condition	n	Pass rate	Mean tokens	Mean LLM calls
C — fixed pipeline	5	100%	240	1.0
D — ARBITER adaptive	5	100%	561	2.0

3.1 What this does and does not show

Every condition solves every task, so **these numbers cannot distinguish the conditions on correctness**. The smoke set is a development fixture, not a benchmark: the tasks are easy enough that a single model call solves them, which is exactly what a smoke set is for. The correct conclusion is that the measurement pipeline works end to end, not that the conditions are equivalent.

What the cost column does already show is the shape of the problem the project exists to study. At identical correctness, **the adaptive condition spent 2.3× the tokens of the single agent** (561 vs 240) because its planner adds a call before any work begins. If that gap does not buy correctness on harder tasks, it is a cost with no return — which is precisely the deflationary finding of Tran & Kiela (2025) that this project is designed to test rather than assume. Phase 3 runs the same comparison on tasks hard enough to separate the conditions.

3.2 The validator caught a defect in our own benchmark

On the first sweep, one task (`smoke_code_3` , run-length encoding) scored **0.75 under every condition** — the same partial score for all five, which is itself a signal: a model failure would vary between conditions, a task failure would not.

Inspecting the trace showed the generated code was **correct**. The task specification asked for the encoded string *only when it is strictly shorter* than the input; for the input `'aaabbc'` the encoding `'a3b2c1'` is six characters against six, so the correct answer is the original string. Our own test asserted the encoded form. **The benchmark task was wrong, not the model**. The task was rewritten unambiguously, the affected runs were deleted, and all five conditions were re-run; all now pass.

This is worth reporting for three reasons. Graded per-test scoring made the defect visible — a binary pass/fail would have shown only "fail". The trace made it diagnosable. And a project whose entire output is a comparison table has to be able to tell "the system is wrong" from "the measurement is wrong", which is the discipline this incident demonstrates.

4. Live deployment

<https://arbiter-gules-two.vercel.app>

The demo is deployed on Vercel as a single Python function serving the same FastAPI application that runs locally, so the hosted demo and the laptop demo cannot drift apart. Fluid compute is enabled, which is what allows one request to stream Server-Sent Events for the length of a run instead of buffering until it ends.

The deployment was verified by running an unseen task against it from a browser. The test-designer agent wrote five tests, the first generated program **failed every one of them (score 0.00)**, the decision policy chose `refine` , and the second attempt passed all five — 2 iterations, 3 model calls, 2,830 tokens, 5.4 seconds. The generated program is executed in a sandboxed subprocess on the host, so the validator result is an execution outcome, not an opinion. The screenshot in Section 5.5 is that run.

Live runs are token-gated. A public URL that executes model-written Python is a remote shell, so it is not offered openly. The demonstration link carries an authorisation token; without it a visitor still gets the full interface, the offline scripted provider and replay of stored runs, but cannot execute code or spend API quota. The gate is covered by automated tests rather than by configuration alone.

5. What was built

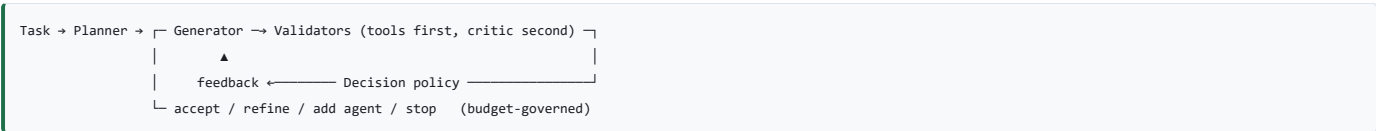
5.1 An agent is a function

No agent framework is used. An agent is a role prompt, one model call, and a typed parse:

```
def generator(task, router, state, feedback=None):
    prompt = build_prompt(task, feedback)
    resp = router.call(prompt, schema=SOLUTION_SCHEMA, state=state)
    return Solution.model_validate_json(resp.text)
```

Different agents differ only in their system prompt. `repair` is told not to trust the previous approach; `critic` is told to review sceptically. Keeping this explicit — rather than inside LangGraph or AutoGen — is deliberate: the orchestration logic is what the project studies, so it must remain readable.

5.2 The orchestrator is a bounded loop



The loop can never run unbounded. It stops on acceptance, 5 iterations, 20 LLM calls, 60,000 tokens, 180 seconds, two iterations without improvement, or a repeated identical output — the last being the most common documented multi-agent failure mode (step repetition, 17.1% of failures in the MAST taxonomy, NeurIPS 2025). Every stop reason is recorded.

5.3 The test-designer agent

Benchmark tasks ship with tests. A task typed by a panel member does not. The test-designer agent derives executable tests from the task description and fixes the function signature, which is then given to the generator — otherwise the tester invents `two_sum` while the generator writes `twoSum` and everything fails for the wrong reason.

Generated tests are validated before use: they must define `check_*` functions, must call the declared entry point, and are rejected if they contain imports, file access or other constructs they have no business using.

Stated limitation: model-written tests are not ground truth. They are used only for live unseen tasks. All benchmark measurement uses the dataset's own tests, and the two are never mixed in results.

5.4 Condition A+ — the baseline that makes the project credible

Most student multi-agent projects compare their system against a single LLM call, declare a win, and ignore that the multi-agent system spent five times the tokens. Condition **A+** removes that confound: the single agent receives **the same token budget** the adaptive condition actually consumed on that task, spent on repeated sampling with majority voting.

A+ is deliberately denied access to the validator. If it could see test results it would be a refinement loop, not a single agent, and the comparison would be rigged.

5.5 Live interface

The web interface streams agent events over Server-Sent Events as they happen. The panel can type any task and watch the roles, validator verdicts, iterations, LLM calls and token count update live. A replay mode re-streams a stored run with original timings, clearly labelled, so the demonstration survives a dead API key or venue wifi.

Below: a live run on the deployed instance. The first attempt scores 0.00, the loop refines, the second attempt scores 1.00 and is accepted. Nothing is scripted — the task was typed into the box.

ARBITER. Adaptive Role-Based Iterative Task Execution & Refinementlive mode ready

Task — type anything, this is not a scripted demo

Evaluate a string arithmetic expression containing non-negative integers and the operators + - * / with correct operator precedence and no parentheses. Division truncates toward zero. Do not use eval.

Familycode — validated by running tests

ConditionB — generator ↔ validator loop

Run

TASKcondition B

Evaluate a string arithmetic expression containing non-negative integers and the operators + - * / with correct operator precedence and no parentheses. Division truncates toward zero. Do not use eval.

TEST_DESIGNER

entry=evaluate_expression, 5 tests

PLAN

baseline B

GENERATORiter 1

1357 chars

PYTHON_EXECiter 1score 0

python_exec score=0.00

DECIDEiter 1

refine

GENERATORiter 2

1215 chars

PYTHON_EXECiter 2score 1

python_exec score=1.00

DECIDEiter 2

6. Engineering decisions worth defending in viva

Decision	Alternative considered	Why this
Role selection from a fixed library	Inventing agents at runtime	Runtime invention is unbounded and unreproducible — it cannot be measured, so it would be a demo, not an experiment
Custom ~400-line orchestrator	LangGraph / AutoGen	The orchestration logic is the object of study; AutoGen is also in maintenance mode
Tools validate, LLM critic advises	LLM-only review	Self-correction without external feedback is known to be unreliable; the critic's verdict is logged separately to measure false accepts
Disk response cache from day one	Add caching later	Re-runs cost nothing and become byte-reproducible; without it the free-tier quota makes the experiment infeasible
Graded score, not pass/fail	Binary pass/fail	Partial credit is what lets the loop detect improvement and stop when stuck
No code execution on public host	Sandbox everywhere	Arbitrary remote code execution is not an acceptable risk for a demo

7. Remaining work

Phase	Dates	Work
Phase 2	11 – 31 Oct	Full benchmark datasets (HumanEval+/MBPP+, Spider, GSM8K); evaluate condition D; cross-model validation experiment
Phase 3	1 – 10 Nov	Frozen benchmark sweep, comparative results, ablation on iteration count and validator type, failure classification, deployment, report
Final	23 – 27 Nov	Report, demonstration, viva

Known risks

API quota is the main risk. A full sweep is several thousand model calls against a free tier, and only one key is currently provisioned — the first sweep already retired that key once on a rate limit, which the router handled by design. Four controls are implemented: the response cache (52% hit rate across the first sweep, so re-runs are largely free), round-robin across keys as further keys are added, a resumable runner that checkpoints after every task and resumes exactly where it stopped, and a reserved demo key the experiment runner never touches so demo-day quota stays fresh.

Majority voting is weaker for code than for math. Two correct programs rarely match character for character, so A+ is a weaker opponent on code than on math. This is reported as a limitation rather than hidden, and it is itself part of the result.

8. Work distribution

Member	Contribution this phase
Gautam Lasgotra (23BCS032)	System architecture, orchestration loop and decision policy, budget governor, provider abstraction with caching and key rotation, sandboxed execution, web interface and streaming, integration
Chirag Attri (23BCS024)	Agent role library and prompt design, structured output schemas, test-designer agent, LLM critic, provider integration
Aniket Kundal (23BCS015)	Benchmark task format and smoke dataset, execution validators, metrics module, resumable experiment runner, test suite

Verification: `pytest -q` → 42 passed, offline, no API key required. Run the demo: `python -m uvicorn arbiter.web.app:app --reload` → `http://127.0.0.1:8000`